

Operators (Study Notes)

Definitions

- **Operator:** A symbol that tells the compiler or interpreter to perform specific mathematical, logical, or relational operations. For example, `+` (addition) and `-` (subtraction) are operators that perform arithmetic actions.
- **Operand:** The data on which an operator acts. For example, in `5 + 3`, the numbers `5` and `3` are operands. The operator `+` combines these operands.
- **Expression:** A combination of operands and operators that the programming language evaluates to a value. An expression consists of one or more operands and zero or more operators. For example, `5 + 3 * 2` is an expression.

Categories of Operators

Operators are grouped into categories based on their functionality. Key categories include:

- **Arithmetic Operators:** Perform mathematical calculations. Examples: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `%` (modulus).
- **Relational (Comparison) Operators:** Compare two operands and produce a boolean result (`true` or `false`). Examples: `==` (equal to), `!=` (not equal to), `>` (greater than), `<` (less than), `>=` (greater than or equal to), `<=` (less than or equal to), and `instanceof` (type check in languages like Java).
- **Logical Operators:** Combine or invert boolean conditions. Examples (in Java/C-like syntax): `&&` (logical AND), `||` (logical OR), and `!` (logical NOT).
- **Unary Operators:** Operate on a single operand. Common ones: `++` (increment), `--` (decrement), unary `+` (positive sign), unary `-` (negative sign), and `!` (logical NOT). The increment/decrement can be prefix (e.g. `++x`) or postfix (e.g. `x++`).
- **Assignment Operators:** Assign or modify values of variables. The basic assignment is `=`. Compound assignments combine an operator with `=`, for example `+=` (add and assign), `-=` (subtract and assign), `*=` (multiply and assign), `/=` (divide and assign), etc.
- **Bitwise Operators:** Operate on the individual bits of integer values. Examples: `&` (bitwise AND), `|` (bitwise OR), `^` (bitwise XOR), `~` (bitwise NOT), as well as bit-shift operators `<<` (left shift), `>>` (signed right shift), and `>>>` (unsigned right shift in Java).
- **Ternary (Conditional) Operator:** A shorthand conditional operator that takes three operands. In Java/C syntax, it is `(condition) ? expr1 : expr2`, which evaluates to `expr1` if the condition is true, otherwise `expr2`.
- **Type Comparison Operator (`instanceof` in Java):** Checks whether an object is an instance of a specific class or interface, returning `true` or `false`. (Shown below with examples.)

Arithmetic Operators

Arithmetic operators perform basic mathematical computations. Examples in Java:

```

public class Main {
    public static void main(String[] args) {
        int division = 5 / 2;
        System.out.println(division);    // integer division: 5/2 = 2

        int mod = 5 % 2;
        System.out.println(mod);        // modulus: 5%2 = 1

        int sum = 3 + 4;
        System.out.println(sum);        // addition: 3+4 = 7

        int subtract = 4 - 3;
        System.out.println(subtract);    // subtraction: 4-3 = 1

        int multiply = 3 * 4;
        System.out.println(multiply);    // multiplication: 3*4 = 12
    }
}

```

- Step-by-step:

- `5 / 2` performs integer division, resulting in `2`. (Any fractional part is discarded.)
- `5 % 2` computes the remainder of division, which is `1`.
- `3 + 4` yields `7`.
- `4 - 3` yields `1`.
- `3 * 4` yields `12`.

The output of the program above is:

```

2
1
7
1
12

```

Relational (Comparison) Operators

Relational operators compare two operands and return a boolean result. In Java (and similar languages), the main relational operators are `==`, `!=`, `>`, `<`, `>=`, `<=`, and `instanceof`. Example usage:

```

public class Main {
    public static void main(String[] args) {
        int a = 4;
        int b = 7;
        System.out.println(a == b);    // Is a equal to b? false
        System.out.println(a != b);    // Is a not equal to b? true
    }
}

```

```

        System.out.println(a > b);    // Is a greater than b? false
        System.out.println(a < b);    // Is a less than b? true
        System.out.println(a >= b);   // Is a greater than or equal to b? false
        System.out.println(a <= b);   // Is a less than or equal to b? true
    }
}

```

- Step-by-step:

- `a == b` checks equality: `4 == 7` is `false`.
- `a != b` checks inequality: `4 != 7` is `true`.
- `a > b`: `4 > 7` is `false`.
- `a < b`: `4 < 7` is `true`.
- `a >= b`: `4 >= 7` is `false`.
- `a <= b`: `4 <= 7` is `true`.

The output is:

```

false
true
false
true
false
true

```

Logical Operators

Logical operators combine boolean conditions. In Java/C: `&&` (AND) is true if **both** operands are true; `||` (OR) is true if **at least one** operand is true; `!` (NOT) inverts a boolean value. Example:

```

public class Main {
    public static void main(String[] args) {
        int a = 4;
        int b = 7;

        // Logical AND (&&)
        System.out.println((a < 3) && (a != b)); // false && true -> false
        System.out.println((a > 3) && (a != b)); // true && true -> true

        // Logical OR (||)
        System.out.println((a < 3) || (a != b)); // false || true -> true
        System.out.println((a > 3) || (a != b)); // true || true -> true
    }
}

```

- Step-by-step (AND examples):

- For `(a < 3) && (a != b)`: `a < 3` is false, `a != b` is true; false && true is false.
- For `(a > 3) && (a != b)`: `a > 3` is true, `a != b` is true; true && true is true.
- For `(a < 3) || (a != b)`: `a < 3` is false, `a != b` is true; false || true is true.
- For `(a > 3) || (a != b)`: `a > 3` is true, `a != b` is true; true || true is true.

The output is:

```
false
true
true
true
```

Unary Operators

Unary operators operate on a single operand. Common unary operators include increment (`++`), decrement (`--`), unary plus (`+`), unary minus (`-`), and logical NOT (`!`). Example in Java:

```
public class Main {
    public static void main(String[] args) {
        int a = 6;
        boolean flag = true;

        // Postfix and Prefix Increment
        System.out.println(a++); // prints 6, then a becomes 7
        System.out.println(++a); // a becomes 8, then prints 8

        // Postfix and Prefix Decrement
        System.out.println(a--); // prints 8, then a becomes 7
        System.out.println(--a); // a becomes 6, then prints 6

        // Logical NOT
        System.out.println(!flag); // flag is true, so !flag is false

        // Unary Minus and Plus
        System.out.println(-a); // a is 6, so -a is -6
        System.out.println(+a); // +a is just 6 (no change)
    }
}
```

- Step-by-step:

- `a++` (postfix increment) prints the current value 6, then increments a to 7.
- `++a` (prefix increment) first increments a to 8, then prints 8.
- `a--` (postfix decrement) prints 8, then decrements a to 7.
- `--a` (prefix decrement) decrements a to 6, then prints 6.

- `!flag` inverts the boolean: since `flag` is `true`, `!flag` is `false`.
- `-a` applies unary negation: `-6`.
- `+a` is a no-op here (remains `6`).

The output is:

```
6
8
8
6
false
-6
6
```

Assignment Operators

Assignment operators set or update the value of variables. Examples: `=`, `+=`, `-=`, `*=`, `/=`. Example in Java:

```
public class Main {
    public static void main(String[] args) {
        int a = 5;
        int variable;

        variable = a;                // simple assignment
        System.out.println(variable); // prints 5

        variable = 0;
        variable += a;                // equivalent to variable = variable + a
        System.out.println(variable); // prints 5 (0 + 5)

        variable -= 3;               // equivalent to variable = variable - 3
        System.out.println(variable); // prints 2 (5 - 3)

        variable *= a;               // equivalent to variable = variable * a
        System.out.println(variable); // prints 10 (2 * 5)

        variable /= a;               // equivalent to variable = variable / a
        System.out.println(variable); // prints 2 (10 / 5)
    }
}
```

- Step-by-step:

- After `variable = a;`, `variable` is `5`.
- After `variable = 0;` and then `variable += a;`, `variable` becomes `0 + 5 = 5`.
- `variable -= 3;` changes `variable` from `5` to `2`.

- `variable *= a;` multiplies `2 * 5`, giving `10`.
- `variable /= a;` divides `10 / 5`, giving `2`.

The output is:

```
5
5
2
10
2
```

Bitwise Operators

Bitwise operators work on the binary representations of integer operands, manipulating individual bits. In Java, these include `&` (AND), `|` (OR), `^` (XOR), and `~` (NOT). Example:

```
public class Main {
    public static void main(String[] args) {
        int a = 4;    // binary 0100
        int b = 6;    // binary 0110

        System.out.println(a & b);    // bitwise AND: 0100 & 0110 = 0100 (4)
        System.out.println(a | b);    // bitwise OR:  0100 | 0110 = 0110 (6)
        System.out.println(a ^ b);    // bitwise XOR: 0100 ^ 0110 = 0010 (2)
        System.out.println(~a);        // bitwise NOT: ~0100 = 1011 (in 2's
        complement: -5)
    }
}
```

- Step-by-step:

- `a & b`: AND each bit of `a` and `b`. For `0100 & 0110`, result is `0100` (decimal 4).
- `a | b`: OR each bit. `0100 | 0110` yields `0110` (decimal 6).
- `a ^ b`: XOR each bit. `0100 ^ 0110` yields `0010` (decimal 2).
- `~a`: bitwise NOT flips all bits of `a`. For a 32-bit signed `int`, `~0100` (binary) becomes `111...1011`, which is the two's complement representation of `-5`.

The output is:

```
4
6
2
-5
```

How Bitwise NOT Works (Example)

To see why `~4` equals `-5`:

- `a = 4` in binary (8-bit for illustration) is `00000100`.
- `~a` flips all bits: `11111011`. In two's complement, the value of `11111011` is calculated as `-5`:
- Evaluate bits: $1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$. This sum is $-128 + -64 + -32 + -16 + 0 + 4 + 2 + 1 = -133$. However, because we started with 8-bit arithmetic, this pattern represents `-5`. (Alternatively, compute two's complement: the two's complement of `00000101` (5) is `11111011` which is `-5`.)
- In short, bitwise NOT on `n` gives `-(n+1)`, so `~4 = -(4+1) = -5`.

Bitwise Shift Operators

Bit-shift operators move the bits of an integer left or right. In Java:

- `<< k` shifts all bits of a number left by `k` positions (filling in zeros on the right). Shifting left by 1 multiplies the number by 2.
- `>> k` is a *signed right shift*: it shifts bits right by `k` positions, filling in with the sign bit (the leftmost bit) on the left. This preserves the sign of the original number (also divides by 2, rounding toward negative infinity).
- `>>> k` is an *unsigned right shift* (Java only): it shifts bits right by `k`, filling with zeros on the left regardless of sign.

- **Examples:**

```
public class Main {
    public static void main(String[] args) {
        int a = 4; // binary 00000100

        System.out.println(a << 1); // left shift by 1: 00001000 (decimal 8)
        System.out.println(a << 2); // left shift by 2: 00010000 (decimal 16)

        System.out.println(a >> 1); // signed right shift by 1: 00000010
        (decimal 2)
        System.out.println(a >> 2); // signed right shift by 2: 00000001
        (decimal 1)
    }
}
```

- Left-shifting `a=4` by 1 yields `8`; by 2 yields `16`.
- Right-shifting `a=4` by 1 yields `2`; by 2 yields `1`.

The output is:

```
8
16
2
1
```

- **Shift Examples with Sign:**

- For a negative number (e.g., `a = -8`, binary `11111000` in 8 bits):
 - `a >> 1` (signed right shift) yields `11111100` (binary) which is `-4`.
 - `a >>> 1` (unsigned right shift) would yield `01111100` (binary) which is `124` (since the sign bit is filled with 0).

Ternary (Conditional) Operator

The ternary operator is a compact `if-else` for expressions. Syntax: `(condition) ? exprIfTrue : exprIfFalse`. Example:

```
public class Main {
    public static void main(String[] args) {
        int a = 4;
        int b = 5;
        int maxVal = (a > b) ? a : b;
        System.out.println(maxVal);
    }
}
```

- **Explanation:** `(a > b) ? a : b` checks if `a > b`. If true, the expression evaluates to `a`; otherwise to `b`. Here `a=4`, `b=5`, so `a > b` is `false`, and `maxVal` becomes `b` (5).
- **Output:** `5`.

Type Comparison Operator (`instanceof` in Java)

The `instanceof` operator checks whether an object is an instance of a specified class or interface, returning a boolean. For example, given a class hierarchy:

```
ParentClass
├─ ChildClass1 extends ParentClass
└─ ChildClass2 extends ParentClass
```

Example usage in Java:

```
public class Main {
    public static void main(String[] args) {
        ParentClass obj = new ChildClass2();
        System.out.println(obj instanceof ChildClass2); // true
        System.out.println(obj instanceof ChildClass1); // false

        ChildClass1 childObj = new ChildClass1();
    }
}
```



```

        System.out.println(childObj instanceof ParentClass); // true

        String val = "hello";
        System.out.println(val instanceof String); // true

        Object unknownObject = new Random();
        System.out.println(unknownObject instanceof ChildClass2); // false
    }
}

```

• **Explanation:**

- `obj instanceof ChildClass2` is `true` because `obj` refers to a `ChildClass2` object.
- `obj instanceof ChildClass1` is `false` since `obj` is not a `ChildClass1`.
- `childObj instanceof ParentClass` is `true` because `ChildClass1` inherits from `ParentClass`.
- `"hello"` is an instance of `String`, so `val instanceof String` is `true`.
- `unknownObject` refers to a `Random` object, which is not related to `ChildClass2`, so that check is `false`.

• **Output:**

```

true
false
true
true
false

```

Operator Precedence and Associativity

- **Precedence:** Determines the order in which operators are evaluated in an expression. Operators with higher precedence are evaluated before lower-precedence ones. For example, multiplication has higher precedence than addition, so in `3 + 4 * 5`, the multiplication `4 * 5` is done first.
- **Associativity:** When two operators have the same precedence, associativity determines the evaluation order (left-to-right or right-to-left).

The table below summarizes common operators (from highest to lowest precedence) and their associativity in languages like Java/C:

Operators (Category)	Examples (Precedence)	Associativity
Parentheses (grouping)	<code>()</code> , <code>[]</code>	Left-to-right
Postfix (unary)	<code>expr++</code> , <code>expr--</code>	Left-to-right
Prefix (unary)	<code>++expr</code> , <code>--expr</code> , <code>+expr</code> , <code>-expr</code> , <code>~</code> , <code>!</code>	Right-to-left
Multiplicative	<code>*</code> , <code>/</code> , <code>%</code>	Left-to-right

Operators (Category)	Examples (Precedence)	Associativity
Additive	<code>+</code> , <code>-</code>	Left-to-right
Bitwise Shift	<code><<</code> , <code>>></code> , <code>>>></code>	Left-to-right
Relational	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>instanceof</code>	Left-to-right
Equality	<code>==</code> , <code>!=</code>	Left-to-right
Bitwise AND	<code>&</code>	Left-to-right
Bitwise XOR	<code>^</code>	Left-to-right
Bitwise OR	<code> </code>	Left-to-right
Logical AND	<code>&&</code>	Left-to-right
Logical OR	<code> </code>	Left-to-right
Ternary	<code>? :</code>	Right-to-left
Assignment	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , etc.	Right-to-left

- **Notes:** Parentheses force evaluation order (highest precedence). Among assignment operators (e.g. `+=`, `=`), evaluation goes right-to-left (for example, in `a = b = 5`, `b = 5` is evaluated first, then `a = (b = 5)`). Ternary `? :` is also right-associative.

Understanding precedence and associativity is crucial for predicting how complex expressions are evaluated. In ambiguous cases, use parentheses to make the intended order explicit.